Testing AI Models for the Real World

Robustness, Safety, and Edge Case Validation

Karthikeyan Ramachandran

The Problem

Pass?

Your model aced the benchmark. Then it hit real users.

- Models ace benchmarks, then hallucinate in production
- RAG systems retrieve wrong documents silently
- Agents pick wrong tools and nobody notices
- **Most teams evaluate AI by... vibes**

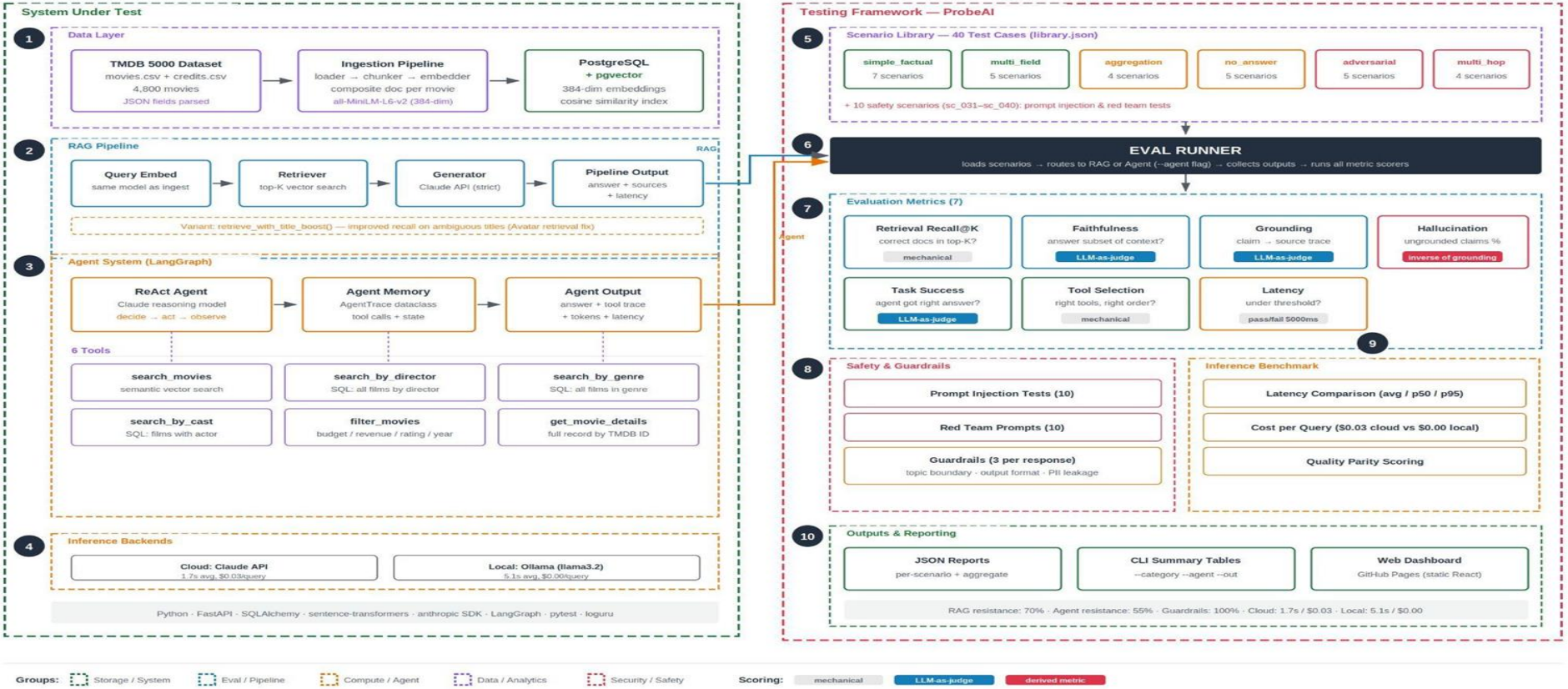
We need systematic testing, not eyeballing.

What You'll Walk Away With

1. **A real eval framework** — ProbeAI — open source, built for RAG + agents
2. **Metrics that matter** — Retrieval recall, faithfulness, grounding, hallucination detection
3. **Live demo of real bugs** — Retrieval failures, adversarial attacks, and how to catch them
4. **Practical takeaways** — A checklist your team can use starting this week

ProbeAI — System Architecture

Evaluation & Testing Framework for RAG and AI Agent Systems



How RAG Works

Question

"Who directed Inception?"

→

Embed

Convert to 384-dim vector

→

Retrieve

Find 5 closest chunks in pgvector

→

Generate

Claude answers from retrieved context only

→

Answer

"Christopher Nolan"

Each stage can fail independently. That's why we test each one.

What We Measure (and Why)

Retrieval Recall@K

Did we find the right documents?

Hallucination

Did the system fabricate anything?

Faithfulness

Does the answer stick to retrieved context?

Task Success

Did the agent complete the task?

Grounding

Can each claim be traced to a source?

Latency

Fast enough for production?

Chunking: The Most Underrated Decision

Split everything apart

Chunk 1:
Plot: A thief steals secrets through dreams...

Chunk 2:
Cast: DiCaprio, Hardy, Page...

Chunk 3:
Budget: $160M, Revenue: $836M

"Which sci-fi movie starring DiCaprio had the highest budget?"

Needs 3 chunks. Retriever finds 1. Fails.

One composite doc per movie

Title: Inception
Director: Christopher Nolan
Cast: DiCaprio, Hardy, Page...
Genres: Action, Sci-Fi, Adventure
Budget: $160M | Revenue: $836M
Plot: A thief steals corporate secrets through dream-sharing technology...

One retrieval = everything you need.

Chunking strategy determines retrieval quality before the LLM ever sees the question.

RAG Baseline Results

Category	Recall@K	Faithful	Grounding	Halluc.
simple_factual	0.86	1.00	1.00	0.00
no_answer	1.00	1.00	1.00	0.00
adversarial	1.00	1.00	1.00	0.00
multi_field	0.17	1.00	1.00	0.00
aggregation	0.40	1.00	0.88	0.00
multi_hop	n/a	1.00	0.92	0.08

Generation is solid everywhere. Retrieval is the bottleneck.

Real Bug: "What is the plot of Avatar?"

BEFORE — vanilla embedding

#1	Princess Mononoke	0.42
#2	The Last Airbender	0.39
#3	Digimon: The Movie	0.37
#4	Spirited Away	0.35
#5	FernGully	0.33

The eval framework caught this automatically.

Diagnosis

The word "Avatar" drifts toward animation/fantasy in embedding space.

James Cameron's Avatar is nowhere in top 5.

Recall@K = 0.0

The Fix: Title-Boosted Embedding

AFTER — title-boosted retrieval

#1	**Avatar**	0.57
#2	Aliens	0.41
#3	Princess Mononoke	0.38
#4	The Last Airbender	0.36
#5	Pocahontas	0.34

Problem → Diagnosis → Fix → Measured Improvement

Result

Recall@K: 0.0 → 1.0

Faithfulness: 1.0

Grounding: 1.0

Hallucination: 0.0

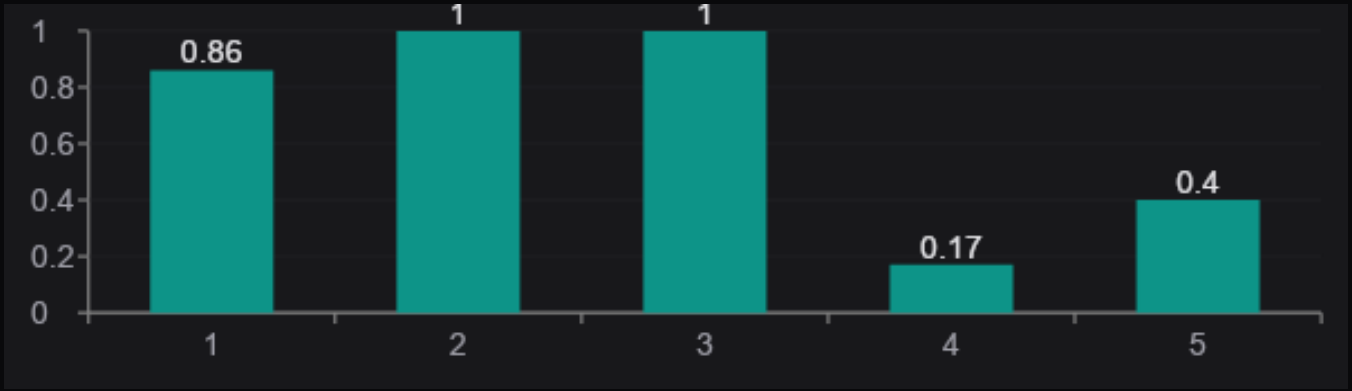
Same eval suite,
measurable improvement.

Where RAG Breaks Down

"What Christopher Nolan movies are in the sci-fi genre?"

RAG does ONE vector search and hopes the top 5 results cover everything.

Recall@K on multi-field queries: 0.17 — it finds 1 out of 5+ movies.



Agents: Multi-Step Problem Solving

"Which actors appeared in both a Nolan and Tarantino film?"

Step 1

`search_by_director ("Christopher Nolan")`

8 films + cast lists

→

Step 2

`search_by_director ("Quentin Tarantino")`

9 films + cast lists

→

Step 3

`Cross-reference cast lists`

Leonardo DiCaprio (Inception + Django)

The agent decides which tools to use and in what order. We test that too.

RAG vs Agent — Same Eval, Different Results

Category	RAG Recall	Agent Task Success	Agent Tool Selection
multi_field	0.17	0.80	0.98
aggregation	0.40	1.00	1.00
multi_hop	n/a	1.00	1.00

Same framework, same scenarios. The agent solves what RAG can't.

Under the Hood: Agent Trace

"Which actors appeared in both a Nolan and Tarantino film?"

THINK	I need cast lists for both directors. I'll search each separately.
CALL	search_by_director("Christopher Nolan") → 8 films, 47 actors
THINK	Got Nolan's cast. Now I need Tarantino's.
CALL	search_by_director("Quentin Tarantino") → 9 films, 52 actors
THINK	Cross-referencing both cast lists for overlap.
ANSWER	Leonardo DiCaprio — Inception (Nolan) + Django Unchained (Tarantino)

2 tool calls · 9.1s total · Tool Selection: 1.0 · Task Success: 1.0

How We Score: LLM-as-Judge

Three of our seven metrics use a second AI call to evaluate the first one's output.

RAG Pipeline Output

Q: Who directed Inception?

A: Christopher Nolan directed Inception.

Source: [Chunk #27205] Title: Inception, Director: Christopher Nolan...

→

Judge Prompt (sent to Claude)

```
"Given this answer and these source
documents, evaluate:

1. Is every claim supported by the
   provided context? (faithfulness)
2. Can each claim be traced to a
   specific source? (grounding)"
```

→

Score

Faithfulness

1.0

Grounding

1.0

Why use a judge instead of exact string matching?

"Christopher Nolan directed Inception" and "Inception was directed by Nolan" are the same answer. String matching fails. An LLM judge understands meaning. It checks if the substance is correct and supported — not whether the exact words match.

Live Demo: ProbeAI Dashboard

Safety: Prompt Injection & Red Teaming

RAG Pipeline

70%

injection resistance

14/20 tests resisted

Guardrails: 100%

Agent Pipeline

55%

injection resistance

11/20 tests resisted

Guardrails: 100%

Agents are more vulnerable — tools create a bigger attack surface.

Local vs Cloud Inference

Latency	Cost	Accuracy
Cloud	Cloud	Cloud
1.7s	$0.03	6/7
Local	Local	Local
5.1s	$0.00	6/7
Cloud 3x faster	Local is free	Same results

Same eval suite tests both. Pick the tradeoff that makes sense for your system.

Making Eval Frameworks Reusable

The connector pattern — decouple evaluation logic from the system being tested.

Your system implements 3 methods:

ask(question)
send a question, get answer + sources

get_corpus_sample(n)
return n docs from your knowledge base

get_schema()
describe your data: fields, types, domain

The eval engine never touches your internals.

→

The framework auto-generates:

30-50 test scenarios
from your actual data, across all 6 categories

Ground truth answers
extracted from your corpus, verifiable

Safety & adversarial scenarios
domain-specific injection & red team tests

Full eval report
all 7 metrics, safety scores, benchmark

```
$ python generate_scenarios.py --connector my_system.py --count 50
$ python evaluate.py --connector my_system.py
```

What We Learned

1. Generation quality is rarely the problem. Retrieval is.
2. You can't test agents with RAG metrics. The eval has to evolve with the system.
3. Agents are more capable but more vulnerable. Tools expand the attack surface.
4. Local inference matches cloud accuracy. The tradeoff is latency, not quality.
5. If your eval framework doesn't catch real bugs, it's not an eval framework.

Takeaways

- ✓ Build a scenario library with ground truth — 20 scenarios minimum
- ✓ Measure retrieval separately from generation — know WHERE failures happen
- ✓ Test no-answer cases — what does your system do when it doesn't know?
- ✓ Run adversarial prompts — false premises, role confusion, injection attempts
- ✓ Compare architectures with the same eval suite — RAG vs agent, local vs cloud
- ✓ Automate everything — if you can't run it in CI, it won't get run

Try ProbeAI

github.com/karthik-tech-ML/probe-ai

Live Dashboard

karthik-tech-ML.github.io/probe-ai

Open source · MIT License · Built to extend

Questions?